

Reusable App Building Process

PlainPath Lessons Learned

Document type: Postmortem + Reusable Framework | Project: PlainPath iOS App | May 2026

1. Original Product Direction

What PlainPath Started As

PlainPath was conceived as a broad AI-powered legal and document intelligence platform. The early vision included multiple tools covering contract review, lease analysis, employment agreements, terms of service, general document Q&A, and possibly document drafting. The premise was to give everyday people the kind of document understanding that previously required a lawyer.

How the Scope Changed

Early development revealed a hard truth: trying to build five mediocre tools is worse than building two excellent ones. The scope was narrowed to two core capabilities:

- 1. Analyze a Document** -- upload any document and receive a plain-English breakdown of key terms, risks, and obligations.
- 2. Contract Review** -- deeper structured review of contracts with clause-level flagging and actionable guidance.

This narrowing happened during development rather than before it, which meant some UI work, navigation structure, and onboarding copy had to be revised mid-build.

Why the 2-Tool MVP Was Stronger

- Each tool could be polished to a high standard rather than all tools being rough.
- The value proposition became easier to communicate in App Store metadata and screenshots.
- The onboarding flow was simpler, reducing friction for new users.
- There were fewer edge cases to handle in the AI prompt layer.
- Review notes and privacy disclosures were cleaner and more accurate.

Lesson: Define the MVP tool set before writing a single line of code. Lock it in writing before design begins.

2. Design and UX Process

What Worked Well

- **Canvas-first exploration** was effective for comparing layout variants side by side before committing to code.
- The core design language -- warm off-white background (`hsl(40 20% 97%)`), steel-blue primary (`hsl(211 39% 49%)`), Inter + Instrument Sans type -- was established early and held consistently.
- The document upload and results display patterns were clean on first implementation and required minimal rework.
- The subscription paywall and RevenueCat integration were designed with a clear visual hierarchy.

What Should Have Been Locked Earlier

- **App icon.** The icon was designed late and not formally approved before being wired into the Xcode project. This led to multiple replacement cycles.
- **Screenshot compositions.** App Store screenshots were not planned until near submission. Producing them at 1290x2796 (iPhone 16 Pro Max) correctly on the first try took more effort than it should have.
- **Navigation structure.** The tab and modal navigation evolved during implementation. Locking navigation architecture at Gate 3 would have prevented mid-build restructuring.

How Visual Assets Affected Submission Readiness

- The blue-X Capacitor placeholder icon appeared in App Store Connect build 48 because the production icon had never been pushed to GitHub. This required a debugging cycle to identify root cause (CI checkout from GitHub, not local repo) and a workflow fix.
 - Screenshots needed to be re-exported at Apple-required pixel dimensions. Screenshots taken at the wrong size are silently rejected during upload.
 - Icon inconsistency across the iOS asset catalog (multiple sizes expected in older Xcode formats vs. single 1024x1024 universal in Xcode 14+) caused confusion about which file mattered.
-

3. Engineering Process

What Was Efficient

- **Capacitor as the iOS bridge** was the right call. A single React/TypeScript codebase deployed to both web and iOS without maintaining two separate codebases.
- **RevenueCat for subscriptions** handled App Store In-App Purchase receipt validation, paywall presentation, and entitlement management without custom server infrastructure.
- **Clerk for authentication** eliminated auth boilerplate and provided a production-grade user management system from day one.
- The **GitHub Actions CI pipeline** (`.github/workflows/ios-testflight.yml`) for automated TestFlight builds was well-structured once complete. Every push to main produces a new signed build.

- The **pnpm monorepo workspace** kept the web app, API server, marketing site, and pitch deck isolated but sharing dependencies cleanly.

What Caused Rework

- **Duplicate iOS folder confusion.** Early in the project there was ambiguity between paths like `ios/App/App/` and `artifacts/plainpath/ios/App/App/`. Any time an asset was placed in the wrong directory, the build used the wrong file silently.
- **Capacitor sync in CI overwriting concerns.** `cap sync ios` was suspected of overwriting the icon. Investigation confirmed it does not touch `AppIcon.appiconset`, but the suspicion caused time lost on a false trail.
- **Icon never pushed to GitHub.** Local commits were checkpoint-saved in the Replit environment but not pushed to the GitHub remote. CI always builds from GitHub, not from the Replit workspace. This gap between "committed locally" and "on GitHub" was not obvious and produced build 48 with the wrong icon.
- **Build number management.** Using `github.run_number` as the build number is clean, but early builds incremented without a successful archive, consuming numbers that can never be reused in App Store Connect.
- **Xcode version pinning.** The `DEVELOPER_DIR` env var to pin Xcode 26.0 required investigation before it produced a stable build. Swift version mismatches between the CI runner's default Xcode and the Capacitor Swift PM binaries caused initial build failures.

Where File Paths Created Friction

Problem	Location	Resolution
Icon committed locally, not pushed	<code>AppIcon-512@2x.png</code>	<code>git push origin master:main</code>
<code>cap sync</code> suspected of overwriting icon	<code>ios-testflight.yml</code> step 6	Confirmed not the cause; added verification step
Xcode version mismatch	CI runner default vs. pinned	<code>DEVELOPER_DIR=/Applications/Xcode_26.0.app/Contents/Developer</code>
App icon filename confusion	<code>AppIcon.appiconset/</code>	Single <code>AppIcon-512@2x.png</code> is the only file Xcode 14+ needs

How Build Verification Should Improve

- Add a CI step that runs **before archiving** to assert: icon is 1024x1024, icon file size is above a minimum threshold (placeholders are small), and `Contents.json` references the correct filename.
 - After export, run `unzip -p App.ipa Payload/App.app/AppIcon*.png | identify -` to confirm the icon embedded in the IPA matches the expected dimensions. This closes the loop between source file and shipped artifact.
 - Never assume a local file change is "done." The definition of done is: **the change is on GitHub and a successful CI build has been verified in App Store Connect.**
-

4. Compliance and App Store Readiness

Privacy Labels

- Privacy nutrition labels in App Store Connect must be completed before submission. PlainPath collects user-uploaded document content and account information; these data types must be declared accurately.
- Mismatch between declared data types and actual API behavior is a common rejection reason. Audit every API call for data collected.

App Privacy Page

- The in-app privacy policy URL must be live and accessible before submission. A dead link or placeholder URL is an immediate rejection.
- The privacy policy must specifically address: data storage location, data retention, third-party services (Clerk, RevenueCat, AI provider), and user data deletion rights.

Subscription and IAP Visibility

- RevenueCat products must be linked to App Store Connect In-App Purchases. Products that exist in RevenueCat but are not approved in App Store Connect will not display to users.
- Subscription pricing, duration, and trial terms declared in the app UI must exactly match what is configured in App Store Connect.
- The "Manage Subscription" link must be accessible from within the app (Settings or Profile screen) per App Store guidelines.

External Purchase Risk

- Any reference to web-based purchasing, links to a website for subscription management, or language about pricing that differs from App Store pricing is a rejection risk under App Store guideline 3.1.1.
- All purchase language in the app must flow through StoreKit / RevenueCat -- never direct users to a URL to subscribe.

Screenshots by Device Type

- Apple requires screenshots for at least one of: iPhone 16 Pro Max (6.9") or iPhone 15 Plus / 14 Plus (6.7").
- Required resolution for 6.9": **1320x2868** or **1290x2796** pixels.
- Screenshots that are even 1px off are silently rejected.
- Screenshots must show the actual app, not mockups or marketing composites.

TestFlight and App Store Connect Workflow

- Every TestFlight build must process (typically 5-15 min after upload) before it is visible in App Store Connect.
- Build numbers cannot be reused. If a CI run increments the number but fails to archive, that number is gone.

- A build can be selected for App Store review only after it passes Apple's automated checks. Selecting a build too early before processing completes is a common mistake.

App Icon Validation

- The icon shown in App Store Connect comes from the 1024x1024 asset in `AppIcon.appiconset`. It is baked into the IPA at archive time.
- The icon shown in App Store Connect's "Included Assets" panel reflects what is in the submitted build -- it cannot be changed without submitting a new build.
- Always verify the icon inside App Store Connect after build processing, not just in the source repo.

Review Notes and Metadata

- Provide a reviewer account (username + password) in the review notes if the app requires login to access any functionality.
 - The app description, keywords, and subtitle must not contain competitor names, irrelevant keywords, or promises the app cannot keep.
 - Age rating questions must be answered conservatively; incorrect answers (e.g., claiming no user-generated content when document uploads are supported) cause rejection.
-

5. Top 10 Bottlenecks

- 1. Icon never pushed to GitHub.** The production icon was committed locally in the Replit environment but the `git push` to GitHub (origin/main) was never executed. Eight CI builds used the wrong icon as a result.
- 2. MVP scope defined too late.** The broad multi-tool vision was not narrowed to the 2-tool MVP until development was already underway, causing UI structural rework.
- 3. Xcode version and Swift compatibility.** Pinning the correct Xcode version on the CI runner (and understanding why the default Xcode version produced Swift binary incompatibility) required multiple failed builds to diagnose.
- 4. Screenshot production left to the end.** App Store screenshots at exact Apple dimensions (1290x2796) were not planned until the final stages. Earlier planning would have made them a parallel-path activity rather than a blocker.
- 5. CI pipeline created from scratch mid-project.** The GitHub Actions workflow was built while the app was being developed, not before. A pre-built, tested CI template would save multiple build cycles.
- 6. False trail on cap sync as icon culprit.** Time was spent investigating whether `cap sync ios` was overwriting the icon before confirming it was the GitHub push gap.
- 7. No end-to-end build verification checklist.** There was no formal checklist to run before calling a build "submission-ready." Items like "verify icon in App Store Connect," "confirm screenshots uploaded," and "check subscription product approval" were discovered reactively.

8. Compliance preparation was not parallelized. Privacy labels, privacy policy URL, subscription configuration, age rating, and review notes could all have been prepared during development. Instead they were addressed sequentially at the end.

9. In-app purchase configuration lag. RevenueCat products and App Store Connect IAP products must be set up and approved before submission. This process has its own timeline and cannot be rushed.

10. No canonical source of truth for asset paths. Ambiguity about which directory was the "real" iOS project folder caused repeated confusion when placing icons, checking archives, and writing CI steps.

6. Top 10 Best Decisions

1. Narrowing to two polished tools. Focusing depth over breadth made the app reviewable, communicable, and completable.

2. Capacitor for iOS. Reusing the React web codebase for iOS avoided maintaining two separate UI codebases, cut development time in half, and allowed web-first testing before touching iOS at all.

3. RevenueCat for subscriptions. Delegating subscription management, receipt validation, and payroll presentation to RevenueCat removed significant complexity and compliance risk.

4. Clerk for authentication. Production-quality auth with zero infrastructure -- users got email login, session management, and account recovery without custom server code.

5. GitHub Actions CI for TestFlight. Automating the build, sign, archive, and upload pipeline meant every push to main produced a testable build. Manual Xcode archiving is error-prone and slow.

6. pnpm monorepo structure. Keeping the web app, API server, marketing site, and pitch deck in a single monorepo with shared type definitions and dependencies prevented drift and duplication.

7. Canvas-first design exploration. Comparing design variants on the canvas before committing to code saved implementation time and led to cleaner UI decisions.

8. Single 1024x1024 universal AppIcon (Xcode 14+). Adopting the modern single-asset icon format eliminated the complexity of maintaining 20+ icon sizes in the asset catalog.

9. Adding CI icon verification step. The post-cap-sync sips verification step added to the workflow means any future placeholder icon regression will fail the build loudly rather than shipping silently.

10. Structured YAML CI workflow with numbered steps. Labeling each CI step clearly (1 through 15) made debugging build failures dramatically faster -- log output immediately identifies which step failed.

7. Reusable App-Building Framework

Gate 1: Product Definition

Purpose: Establish what the app does and who it is for before any design or code begins.

Required deliverables:

- One-paragraph product description (what it does, who it's for, what problem it solves)
- Competitive landscape: 3 closest alternatives and why this app is different
- Revenue model confirmed: free, freemium, or subscription
- Platform targets confirmed: iOS only, iOS + web, or all platforms

Required evidence:

- Written product brief signed off by decision-maker
- Confirmed App Store category

Must not change after this gate:

- Core problem being solved
- Primary target user

Pass criteria: All deliverables complete and agreed upon before Gate 2.

Fail criteria: Proceeding to design without a written brief.

Gate 2: MVP Scope Lock

Purpose: Define the exact feature set that ships in v1.0. Everything else is v1.1 or later.

Required deliverables:

- Numbered list of v1.0 features (max 5 core features)
- Numbered list of explicitly excluded features (prevents scope creep)
- User flow diagram for each core feature
- Data model sketch (what data is stored, where, and by whom)

Required evidence:

- Written scope document
- Confirmed tech stack (framework, auth provider, payment provider, AI API)

Must not change after this gate:

- Feature list for v1.0
- Tech stack choices

Pass criteria: Every team member can describe the app in one sentence and name every v1.0 feature.

Fail criteria: Any ambiguity about what is or is not in v1.0.

Gate 3: UX/Canvas Approval

Purpose: Approve the full UX before any production code is written.

Required deliverables:

- Wireframes or high-fidelity mockups for every screen
- Navigation architecture diagram (tabs, modals, stack navigation)
- Empty state, loading state, and error state designs for all key views
- Onboarding and paywall flow designs

Required evidence:

- Screenshots of all approved screens saved to a /design-approved/ folder
- Written approval of navigation structure

Must not change after this gate:

- Screen count and navigation structure
- Paywall placement and design
- Onboarding flow

Pass criteria: Every screen is accounted for in the mockups and the navigation structure is unambiguous.

Fail criteria: Any screen without an approved mockup.

Gate 4: Design System Lock

Purpose: Lock all visual tokens before implementation begins.

Required deliverables:

- Color palette (primary, secondary, background, surface, text, error, success)
- Typography system (font families, sizes, weights, line heights)
- Spacing scale (4px or 8px base grid)
- Component library: buttons, inputs, cards, modals, loading states
- App icon (final, 1024x1024, approved)

Required evidence:

- Design tokens defined in code (CSS variables, Tailwind config, or similar)
- App icon committed to /assets/icon-1024.png in the repo root
- Contents.json for AppIcon.appiconset confirmed correct

Must not change after this gate:

- App icon
- Color palette
- Typography choices

Pass criteria: Design system is in code, app icon is in the repo and verified as 1024x1024.

Fail criteria: App icon not finalized, colors or fonts still in flux.

Gate 5: Core Feature Implementation

Purpose: All v1.0 features are implemented and functional against real data.

Required deliverables:

- All features from Gate 2 scope list are working
- No mock data in production paths
- Authentication working end-to-end (real user creation, session persistence)
- Subscription flow working (real product from App Store Connect, RevenueCat entitlements)
- All API calls going to production or production-equivalent staging

Required evidence:

- Walkthrough video of each core feature
- Error handling confirmed (bad input, network failure, API error)

Must not change after this gate:

- Core feature behavior
- API contract between frontend and backend

Pass criteria: All features demonstrable with real data, no placeholder content.

Fail criteria: Any core feature using mocked or hardcoded data.

Gate 6: Mobile/Responsive QA

Purpose: Confirm the app looks and works correctly on actual devices before compliance work begins.

Required deliverables:

- Tested on at least: iPhone SE (small), iPhone 16 Pro (standard), iPhone 16 Pro Max (large)
- No layout overflow, clipped text, or untappable buttons on any target device
- Safe area insets (notch, Dynamic Island, home indicator) handled correctly
- Keyboard behavior correct (inputs scroll into view, keyboard dismisses on tap)
- Dark mode tested (either supported or explicitly disabled)

Required evidence:

- Screenshots from each test device at key screens
- List of known issues with status (fixed / accepted / deferred)

Must not change after this gate:

- Layout structure of existing screens
- Core navigation UX

Pass criteria: No P1 or P2 layout bugs on any target device.

Fail criteria: Any text clipped, button unreachable, or layout broken on any target device.

Gate 7: Compliance and Privacy QA

Purpose: Ensure the app is legally and policy-compliant before producing App Store assets.

Required deliverables:

- Privacy policy live at a permanent public URL
- Privacy policy covers: data collected, storage location, third-party services, data retention, user deletion rights
- App Store Connect privacy nutrition labels completed and matching actual behavior
- Age rating questionnaire completed accurately
- Terms of service live at a permanent public URL
- All external links in the app confirmed working
- "Manage Subscription" link accessible in-app
- No external purchase links or off-App-Store payment references

Required evidence:

- Privacy policy URL accessible from outside the app
- Screenshot of completed Privacy section in App Store Connect

Must not change after this gate:

- Data collection behavior (changes require re-filing privacy labels)
- In-app purchase flow

Pass criteria: All compliance items checked. Legal review (even informal) complete.

Fail criteria: Privacy policy not live, privacy labels incomplete, subscription management not in-app.

Gate 8: App Store Asset Preparation

Purpose: Produce all visual assets required for App Store submission before opening App Store Connect.

Required deliverables:

- App icon confirmed inside a processed TestFlight build in App Store Connect (not just in source files)
- Screenshots at exactly required dimensions:
 - iPhone 6.9" (1320x2868 or 1290x2796) -- required
 - iPhone 6.7" (1320x2868 or 1290x2796) -- optional but recommended
 - iPad 13" (2064x2752) -- required if iPad is supported
 - At least 3 screenshots showing core user value
 - App Store description written (up to 4000 chars)

- Subtitle written (up to 30 chars)
- Keywords written (up to 100 chars, comma-separated)
- Support URL live and working
- Marketing URL (optional, but should be prepared)

Required evidence:

- All screenshots exported at correct pixel dimensions (verify with file info)
- App icon visible in App Store Connect Included Assets panel of a processed build

Must not change after this gate:

- App Store description, subtitle, keywords
- Screenshots

Pass criteria: All assets uploaded to App Store Connect, no validation warnings.

Fail criteria: Any screenshot at wrong dimensions, app icon showing as placeholder in App Store Connect, description not written.

Gate 9: iOS Build/TestFlight Pipeline

Purpose: Confirm the automated build pipeline produces correct, signed, uploadable builds every time.

Required deliverables:

- CI workflow produces a new build on every push to main
- Build is signed with Distribution certificate and App Store provisioning profile
- Build number auto-increments (no duplicates)
- Upload to TestFlight succeeds without manual intervention
- CI log includes: Xcode version, Swift version, icon verification result, archive success, export success, upload success

Required evidence:

- At least 3 consecutive successful CI builds in App Store Connect
- CI log showing icon verification step passed

Must not change after this gate:

- CI workflow file
- App bundle ID
- Signing certificate or provisioning profile (unless renewal required)

Pass criteria: Three clean CI builds visible in App Store Connect with correct icon.

Fail criteria: Any build showing placeholder icon, failed archive, or failed upload.

Gate 10: Final Submission Audit

Purpose: A complete pre-submission checklist completed by a fresh set of eyes.

Required deliverables:

- All Gate 1-9 pass criteria confirmed
- App version number set correctly in Xcode (CFBundleShortVersionString)
- What's New text written (for update submissions)
- Review notes written (include test account credentials if login required)
- App Store Connect submission form completed: release type (manual or automatic), phased rollout decision, age rating confirmed
- Final TestFlight build selected for submission (not a development or debug build)

Required evidence:

- Completed submission checklist (this document, printed or saved)
- "Ready for Review" status confirmed in App Store Connect

Pass criteria: Every item on the checklist is checked. No open questions.

Fail criteria: Submission attempted with any checklist item incomplete.

Gate 11: App Review Response Handling

Purpose: Respond to Apple's feedback quickly and correctly.

Required deliverables:

- Review outcome monitored daily (App Store Connect and/or email notifications)
- Rejection reasons documented with root cause analysis
- Response plan created within 24 hours of rejection
- Fix implemented, tested, and resubmitted within the CI pipeline -- no ad-hoc manual Xcode uploads
- All rejections and resolutions logged for future reference

Required evidence:

- Written log of each rejection with: rejection reason, root cause, fix applied, resubmission date

Must not change after this gate (without re-testing):

- Any feature or behavior that was cited in a rejection

Pass criteria: App reaches "Ready for Sale" status.

Fail criteria: Resubmitting without fully addressing the rejection reason.

Gate 12: Post-Launch Monitoring

Purpose: Confirm the live app is working correctly for real users after launch.

Required deliverables:

- Crash reporting active (Sentry, Firebase Crashlytics, or equivalent)
- Subscription analytics active in RevenueCat dashboard
- User feedback channel active (support email, in-app feedback, or App Store review monitoring)
- First-week metrics reviewed: downloads, crashes, subscription conversion, churn
- Post-launch retrospective written (what surprised us, what to do differently)

Required evidence:

- Crash rate below 1% within first 7 days
- At least one successful subscriber processed by RevenueCat

Pass criteria: No P1 crashes in production, subscription flow confirmed working with real users.

Fail criteria: Crash rate above 1%, subscription purchases failing, support inbox overwhelming.

8. Build Rules for Future Apps

These are non-negotiable rules, not guidelines.

Rule 1 -- Feature freeze is a real freeze.

No new features are added after Gate 9 (iOS Build Pipeline). Bug fixes only. Any feature added after Gate 9 requires re-running Gates 6, 7, and 8.

Rule 2 -- Verify assets in the archive, not in the source.

The source file being correct does not mean the archive is correct. After every build, open App Store Connect, process the build, and visually confirm the app icon and at least one screenshot are correct in the "Included Assets" panel.

Rule 3 -- Screenshots are verified at pixel level before upload.

Use `file` or `identify` to confirm screenshot dimensions before upload. A screenshot that is even 1px off the required dimension will be silently rejected by App Store Connect with no clear error.

Rule 4 -- The CI pipeline is the only path to TestFlight.

No manual Xcode archives. No drag-and-drop uploads to Transporter. Every build goes through the CI workflow. This ensures reproducibility, correct signing, and the icon verification step runs every time.

Rule 5 -- "Committed" and "pushed to GitHub" are not the same thing.

A commit exists in the local or Replit environment until explicitly pushed to the GitHub remote. CI builds from GitHub. Verify with `git log --oneline origin/main...HEAD` that no local-only commits exist before calling a build cycle complete.

Rule 6 -- App Store Connect, the app binary, and the marketing metadata must stay synchronized.

If the app description says "three tools," the app must have three tools. If the privacy label says "no usage data collected," the app must not collect usage data. Every change to one must trigger a review of the others.

Rule 7 -- Compliance items have their own lead time.

Privacy policy hosting, IAP product approval in App Store Connect, and subscription pricing approval each have timelines outside your control. Start them at Gate 2, not Gate 9.

Rule 8 -- The app icon is locked at Gate 4 and never touched again.

Changing the app icon after development begins wastes CI cycles, risks the wrong icon shipping, and requires re-verifying all marketing assets. Lock it early, commit it to git, push it to GitHub, and do not change it.

Rule 9 -- Every bottleneck gets a root cause, not a workaround.

When something goes wrong (wrong icon in build, CI failure, screenshot rejection), find the actual root cause before applying a fix. Workarounds that mask root causes create the same problem again in the next project.

Rule 10 -- Each gate is documented before moving to the next.

A one-paragraph written summary of what was completed and confirmed at each gate creates an audit trail and prevents "I thought we already handled that" moments.

9. Summary

PlainPath went from idea to App Store submission. That is the most important result. The app works, subscriptions work, authentication works, and the iOS build pipeline is automated and reliable.

The things that slowed us down were almost entirely process gaps, not technical limitations:

- Assets not verified in the final artifact (only in source)
- Scope defined during development instead of before it
- Compliance work deferred to the end instead of parallelized
- The gap between "local commit" and "pushed to GitHub" not treated as meaningful

The reusable framework above is designed to close each of those gaps before the next project starts. The 12-gate structure converts reactive debugging into proactive quality control.

The single most important habit to develop: After every significant build -- open App Store Connect, find the processed build, open "Included Assets," and look at the icon with your own eyes. That 30-second check would have caught the placeholder icon issue immediately on build 1 rather than build 48.

Document version: 1.0

Last updated: May 2026

Based on: PlainPath v1.0 App Store submission